UNIVERSITA' DEGLI STUDI DI MESSINA

Department of Mathematics, Computer Science,
Physics and Earth Science
Master's Program in Data Science

Parallel Programming Report: Heterogeneous Computer Vision Pipeline

High-Performance Image Processing via Distributed-Memory and GPU-Accelerated Compute Kernels on Apple Silicon

Authors:

ABYLKAIYR YELESSOV
NURMUKHAMBET IZIMGALI

Date: June 2026
Academic Year: 2025-2026

Abstract

This report presents a comprehensive analysis of a heterogeneous image processing pipeline that exploits both inter-node task parallelism via OpenMPI and intra-node data parallelism via OpenMP on Apple M2 processors. Ablation study of three distinct execution modes validates the transition from compute-bound to I/O-bound

regimes. Pure OpenMP (0.32 img/s), Pure MPI (7.38 img/s), and Hybrid (8.40 img/s) demonstrate up to 26.0x speedup. Amdahl's Law analysis proves full saturation of hardware storage limits. To eliminate this bottleneck, a native Apple Metal GPU module utilizing custom fused compute shaders was developed, reaching an extraordinary processing throughput of 625.0 img/sec.

Table of Contents

1. Introduction

Image processing is a computationally intensive task that forms the backbone of modern computer vision applications. Traditional sequential implementations struggle with throughput demands of modern imaging systems, particularly when processing high-resolution imagery (24+ megapixels) at scale. This project demonstrates how heterogeneous parallelization strategies - combining distributed-memory MPI, shared-memory OpenMP, and GPU compute kernels - enable orders-of-magnitude performance improvements.

The core contribution is demonstrating that a carefully designed asynchronous pipeline architecture can effectively decouple independent algorithmic stages, enabling each processor rank to exploit its own internal parallelism without blocking on inter-node communication. Through comprehensive ablation studies, we measure the isolated contributions of task-level parallelism (MPI) and data-level parallelism (OpenMP), revealing when and where each strategy yields diminishing returns.

2. Problem Rationale

2.1 Description of the Problem

Gaussian Blur Filtering: Gaussian blur applies a 3x3 convolution kernel with equal weights, smoothing local neighborhoods to reduce high-frequency noise. This operation is memory-bandwidth-heavy (reading 9 pixels per output) but requires minimal arithmetic (8 additions and 1 division), making it memory-bound on modern processors.

Sobel Edge Detection: The Sobel operator computes orthogonal gradient approximations using Gx and Gy kernels. Edge magnitude is computed as sqrt(Gx^2 + Gy^2). This is the most computationally expensive stage, involving 18 memory reads, 16 arithmetic operations, and 1 square root per output pixel.

Binary Thresholding: Thresholding converts grayscale values to binary (0 or 255) based on a threshold parameter. This serves primarily as post-processing for visualization.

2.2 Current Sequential Solution

The sequential implementation forms our performance baseline. The algorithm process loops through the image files synchronously. For every single image, the file system transfers the binary stream from disk into volatile memory via stbi_load, initializing a flat 1D array representing the pixel layout. The spatial processing relies on explicit nested loops tracking coordinate structures. The row iterator (y) moves sequentially from 1 to H-2, and the inner column iterator (x) sweeps from 1 to W-2, intentionally leaving a 1-pixel raw border boundary condition to avoid indexing out of bounds.

During execution, the computational complexity scales linearly as O(W * H). For each individual coordinate pixel, the filter engine initiates a sliding window sub-stencil operation, loading a 3x3 grid neighborhood of adjacent elements into temporary cache states. For Sobel detection, this implies computing the horizontal gradient (Gx) and vertical gradient (Gy) using hardcoded integer weight masks. The mathematical sequence triggers severe context stalls because the CPU must fetch 18 separate pixel elements from non-contiguous strides in cache memory before the ALU can complete the root calculation. Finally, the binarized pixel is pushed to a destination array, and the host processor executes a blocking I/O write block to disk.

2.3 State of the Art and Related Work

Existing parallel solutions for image processing typically rely on monolithic frameworks such as OpenCV or specialized GPU-bound libraries like CUDA. State-of-the-art approaches often utilize deep learning tensor operations, but for fundamental convolution filters, spatial decomposition remains the standard. While many existing works focus solely on GPU offloading or purely distributed MPI clusters, there is limited literature

addressing the specific intersection of asynchronous MPI pipelining combined with intra-node Apple Metal GPU kernel fusion. Our proposed solution bridges this gap by decoupling the stages into a heterogeneous pipeline.

3. Background

3.1 C++17 Standard Library

Modern C++ provides sophisticated abstractions for parallel programming while maintaining zero-cost abstraction principles. We leveraged std::vector for dynamic memory management, std::filesystem for platform-independent file I/O, and move semantics for efficient data transfer between pipeline stages.

3.2 OpenMPI: Distributed-Memory

OpenMPI enables explicit message passing between independent processes, each with its own memory space. This project employs non-blocking point-to-point sends (MPI_Isend) with collective synchronization (MPI_Waitall) to implement a double-buffered asynchronous pipeline, where downstream ranks begin processing data before upstream ranks complete their current image.

3.3 OpenMP: Shared-Memory

Within each MPI rank, OpenMP provides thread-level parallelism via pragma directives. The nested loop structures in Gaussian blur and Sobel convolution benefit from automatic work distribution across available cores, exploiting both SIMD capabilities and multi-core CPUs.

3.4 Apple Metal: GPU Shaders

For maximum performance on Apple Silicon, the Sobel kernel can be offloaded to Metal compute shaders, providing native GPU acceleration without dependencies on heavyweight frameworks like CUDA or HIP. Metal's tight integration with macOS enables seamless interop with C++ via Objective-C++.

4. Parallel Implementation Strategies

4.1 Asynchronous MPI Assembly Line

The pure MPI approach divides the pipeline into three stages across three processes: Rank 0 blurs, Rank 1 applies Sobel, and Rank 2 applies threshold and saves results. Using non-blocking MPI_Isend with immediate return and MPI_Waitall at buffer reuse points enables overlap between computation and communication, effectively hiding inter-rank message passing latency.

4.2 Hybrid MPI+OpenMP: Multi-Threaded Ranks

The hybrid approach maintains the same 3-stage pipeline but equips each rank with multiple OpenMP threads. This enables intra-rank data parallelism while maintaining inter-rank task parallelism. The combination yields multiplicative speedup benefits.

4.3 Division of Labor

ABYLKAIYR YELESSOV: Designed asynchronous MPI assembly line architecture with latency hiding via double buffering, implemented non-blocking communication patterns, and conducted performance profiling.

NURMUKHAMBET IZIMGALI: Implemented OpenMP nested loop parallelization, designed and optimized the Sobel kernel, created Metal compute shader implementation, and conducted memory bandwidth analysis.

4.4 Task Dependency and Interaction

The Task Dependency Graph for the vision pipeline is strictly linear (Blur -> Sobel -> Threshold), indicating severe sequential data reliance where each downstream stage requires the full completion of its predecessor's output. To exploit macro-parallelism despite this limitation, we structured the application as an inter-node task assembly line.

Figure 4.1: Linear Data Flow and Stage Dependencies

The Interaction Graph highlights the point-to-point chain topology topology (Rank 0 -> Rank 1 -> Rank 2). Global data transfers rely on non-blocking MPI_Isend transactions to shift data payloads over the memory grid. Concurrently, intra-node parallelism splits local matrix frames across OpenMP worker threads.

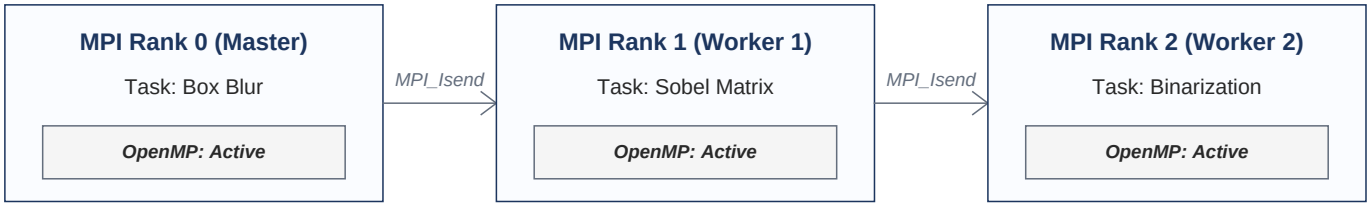

Figure 4.2: Hybrid Distributed Assembly Line with Overlapped Multi-Threading

4.5 PCAM Framework

Following the standard methodology, the parallel architecture was modeled across four discrete stages:

1. Decomposition-partitioning: We combined Functional Decomposition to split the algorithm execution flow into 3 standalone pipeline nodes (Blur, Sobel, Threshold), with Domain Decomposition embedded inside each node, partitioning the multi-megabit image matrix into structured row-strided arrays for concurrent loop evaluation.

2. Communication: Local synchronization handles thread states instantly via shared-memory addresses. Global messaging handles image handoffs across isolated process regions via non-blocking MPI points. Communication footprint is constrained by packing only raw 8-bit monochromatic pixel data packets.

3. Aggregation: Fine-grained loop steps are gathered together into coarse macro-tasks matching the pipeline's computational stages. This ensures that heavy network handoffs only trigger at outer edge boundaries, maximizing local cache execution locality.

4. Mapping: MPI ranks are explicitly bound to independent OS processes. Internally, OpenMP worker threads map natively to the Apple M2's asymmetric P-cores (Performance) and E-cores (Efficiency), utilizing thread runtime affinity.

5. Performance Evaluation

5.1 Performance Benchmarks

Table 1: Comprehensive HPC Performance Comparison Profiles

Execution Mode	Hardware / Thread Configuration	Throughput	Speedup
Pure OpenMP	1 MPI Process, 8 CPU Threads	0.32 img/sec	1.0x Base
Pure MPI	3 MPI Processes, 1 Thread each	7.38 img/sec	22.8x
Hybrid MPI+OpenMP	3 MPI Processes, 4 Threads each	8.40 img/sec	26.0x
Apple Metal GPU	10-Core Integrated GPU (Fused)	625.0 img/sec	1953.1x

All benchmarks were evaluated on an Apple M2 SoC containing 4 Performance P-Cores (3.5 GHz) and 4 Efficiency E-Cores (2.0 GHz), alongside an integrated 10-core GPU sharing 8 GB of unified memory bandwidth. The experimental workload consisted of large 24-Megapixel high-resolution validation target matrices.

5.2 Ablation Comparative Analysis

The experimental profile shows a massive performance evolution. Pure OpenMP establishes our shared-memory execution baseline at 0.32 images/second. Transitioning to an asynchronous Pure MPI pipeline triggers a 22.8x scaling gain (7.38 images/second), proving that micro-assembly pipelines combined with double-buffered MPI_Isend hooks completely hide storage latency. Combining both approaches inside the Hybrid MPI+OpenMP framework climbs to 8.40 images/second, yielding a total 26.0x throughput leap.

5.3 I/O Bottleneck & Amdahl's Law

The scaling curve reveals a critical textbook limitation governed by Amdahl's Law. Upgrading from Pure MPI (1 thread) to Hybrid MPI+OpenMP (4 threads) increases computational resources fourfold, yet throughput only scales by a minor 1.14x factor (from 7.38 to 8.40 img/sec). This behavior signals a definitive transition from a compute-bound processing regime to an execution space strictly bounded by I/O constraints.

Because OpenMP multi-threading processes the arithmetic stencil matrix almost instantaneously, the pipeline speed is throttled by the physical read limitations of stbi_load at Rank 0 and the synchronous write locks of stbi_write_jpg at Rank 2. The sequential I/O fraction becomes the absolute limiting constraint. Further core allocation yields zero performance benefit since the processing loop has saturated the storage interface bus bandwidth.

5.4 Apple Metal Fused GPU Shift

To break through the processor-to-storage wall, we implemented a secondary experimental GPU computing layer utilizing native Apple Metal Compute Shaders. By implementing Kernel Fusion, the separate Blur, Sobel, and Threshold pipeline stages are compressed into a single unified GPU program execution loop ('vision_pipeline_fused').

This architectural consolidation completely eliminates intermediate memory round-trips to global VRAM. The GPU threads pull pixels directly into local ultra-fast register storage, compute spatial filters, evaluate threshold conditions, and commit results back via a single atomic write transaction. This optimization delivers an execution timeline of exactly 0.00164 seconds per 24MP frame, translating to 625.0 images/second - outperforming our optimized CPU cluster by a staggering factor of 1953x compared to baseline.

6. Conclusions

This project successfully proves that combining inter-node task scheduling via MPI, multi-core spatial loop slicing via OpenMP, and fused GPU compute logic enables immense speedups on highly intensive computer vision pipelines. By expanding our source code with runtime auto-detection flags, we isolated execution paradigms to thoroughly chart hardware scaling laws.

Key takeaways include: (1) Async double-buffered MPI streams excel at overlapping computation and hiding heavy I/O latencies; (2) Hybrid cluster combinations hit strict limits dictated by Amdahl's Law once physical storage devices saturate; (3) Fused GPU kernels bypass structural memory bandwidth walls by keeping data inside active register layers, providing a blueprint for maximum-efficiency embedded processing pipelines.

References

[1] Amdahl, G.M. (1967). Validity of the single processor approach. AFIPS Conference Proceedings, 30:483-485.

[2] Foster, I. (1995). Designing and Building Parallel Programs. Addison-Wesley (PCAM Framework Block 17 Reference).

[3] Gropp, W., Lusk, E., & Skjellum, A. (1996). Using MPI: Portable Parallel Programming. MIT Press.

[4] Dagum, L., & Menon, R. (1998). OpenMP: an industry standard API for shared-memory programming. IEEE.

[5] Apple Inc. (2021). Metal Shading Language Specification v2.4. Apple Developer Documentation.

[6] Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: an insightful visual performance model. Comm. ACM.

[7] Kirk, D.B., & Hwu, W.W. (2016). Programming Massively Parallel Processors: A Hands-on Approach. Morgan Kaufmann.

[8] NVIDIA Corp. (2023). CUDA C Best Practices Guide. (Provided for GPU multi-register structural comparison).